SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5

14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

QUESTION 1: Online Banking Fund Transfers: ECP, BVA and Random Testing

The repeated failures at minimum and maximum transfer limits indicate that the existing tests probably use typical values and do not adequately test boundaries, invalid inputs, or combinations of account conditions. A test strategy based mainly on a few normal transactions cannot expose off-by-one errors, incorrect comparisons, or limit-checking defects.

Equivalence Class Partitioning (ECP) divides transfer amounts into groups expected to behave alike. If the permitted amount is L to U, the classes are: below L (invalid), L through U (valid), and above U (invalid). Other classes should cover non-numeric input, blank amount, insufficient balance, blocked account, and unsupported currency where applicable.

Test class	Example input	Expected result
Below minimum	L − smallest currency unit	Reject with clear validation message
Minimum boundary	L	Accept if other rules pass
Normal valid amount	Between L and U	Transfer succeeds
Maximum boundary	U	Accept if other rules pass
Above maximum	U + smallest currency unit	Reject
Insufficient funds	Amount within limits but greater than available balance	Reject; no debit or credit
Invalid format	Blank, letters, negative amount	Reject safely

Boundary Value Analysis (BVA) should test L−1, L, L+1 and U−1, U, U+1 (using the smallest allowed currency unit). Also test zero, negative values, decimal precision, and balance-related boundaries. Verify both the displayed result and database effects: exactly one debit, exactly one credit, correct balance, and a transaction record.

Random testing can discover unexpected failures, so it should be retained as supplementary fuzzing with reproducible seeds, but not used as the main strategy. Random values may rarely hit exact boundaries or important business combinations.

Optimized approach: derive ECP classes, apply BVA to every numeric limit, add decision-table tests for account status/balance/transfer type, run regression tests after fixes, and use random/fuzz tests for unexpected formats. Prioritize money integrity, audit logs, duplicate requests, timeouts, and rollback behaviour. This combines systematic defect detection with exploratory breadth.

QUESTION 2: Healthcare Application: Coverage and Control Flow Graph

Critical post-deployment defects suggest that testing may have measured only whether selected statements executed, rather than whether decisions and important execution paths behaved correctly. Healthcare software also needs risk-based tests for missing data, invalid values, permissions, error handling, and safety-related outcomes.

Conceptual control-flow graph (CFG):
START → Read patient data → [Data valid?]
No → Show validation error → END
Yes → [User authorised?]
No → Deny access and log event → END
Yes → Process result → [Processing successful?]
No → Record error / safe fallback → END
Yes → Save result and audit record → END

The missing paths commonly include invalid or incomplete patient data, unauthorised access, processing exceptions, database failure, and successful processing. Tests must exercise both

True and False outcomes of every decision, not just the happy path.
Statement coverage checks whether each executable statement ran; it can miss an untested decision outcome.

Branch coverage checks each decision outcome (True and False) and is stronger for conditional logic.

Path coverage checks sequences of branches. Full path coverage may be impossible when loops or many combinations exist, so critical paths and bounded loop paths must be selected by risk.

Improved strategy: construct and review the CFG; identify independent paths; create tests for each branch; include boundary, invalid, exception, permission, and recovery cases; measure statement and branch coverage; and use targeted path coverage for safety-critical workflows.

Add integration tests with the database and audit system, regression tests for every fixed defect, and traceability from requirements to tests. Coverage percentages alone do not prove correctness, so expected clinical/business outcomes must also be checked.

QUESTION 3: E-commerce Price, Quantity and Discount Combinations

Testing each input separately is insufficient because a defect may occur only when values interact—for example, a large discount with a high quantity or a zero price. The current approach should be expanded from isolated examples to structured combination testing.

Use ECP for price (negative, zero, valid positive, above allowed maximum), quantity (negative, zero, valid, above stock/limit), and discount (negative, zero, permitted range, above 100% or configured cap). Use BVA at each limit, such as minimum price, maximum quantity, and maximum discount, including one unit below, at, and above the boundary.

Condition	Representative cases	Expected behaviour
Price	−1, 0, minimum valid, normal, maximum, above maximum	Reject invalid price; calculate valid price correctly
Quantity	−1, 0, 1, stock limit, stock limit + 1	Reject invalid quantity; prevent overselling
Discount	−1%, 0%, allowed maximum, above maximum	Reject out-of-range discount; cap only if specification says so
Combination	Valid price + valid quantity + max discount	Correct total and rounding
Invalid interaction	Zero price + discount; quantity beyond stock + discount	Apply business rules consistently; no negative total

Decision table testing should represent rules such as item available/not available, customer eligible/not eligible, discount valid/invalid, and quantity within stock/out of stock. Each meaningful rule column becomes a test. If the full Cartesian product is too large, use risk-based pairwise testing for broad combinations, then explicitly test high-risk three-way interactions and business rules.

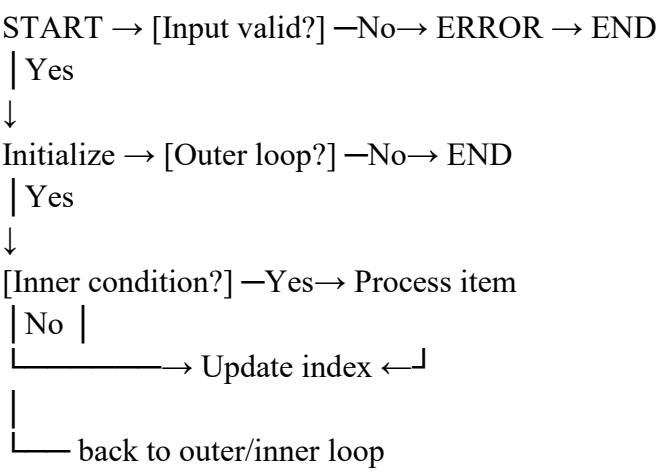
For each case, verify subtotal = price × quantity, discount calculation, tax and shipping rules, rounding, final total, and inventory update. The revised design improves completeness by combining partitioning, boundary tests, and decision rules rather than relying on a small set of ordinary examples.

QUESTION 4: Nested Conditions and Loops: Independent Path Testing

Partial path coverage often results from an incomplete CFG, omitted loop exits, ignored exception edges, or tests that execute only the normal route. Nested conditions multiply possible routes, and loops can create many or infinitely many paths. The aim is to test every independent basis path and all critical condition outcomes, not to claim impractical exhaustive coverage.

Recommended CFG construction process: identify entry and exit nodes; represent each statement or decision; draw True/False edges; include loop-back edges, loop exits, and exception/early-return edges; then review the graph against the source code.

Illustrative flow:



Use cyclomatic complexity as a guide to the number of linearly independent paths: $V(G) = E - N + 2P$, where E is edges, N is nodes, and P is the number of connected components (usually 1). For a single connected graph, $V(G) = E - N + 2$; it can also be calculated as the number of decision nodes + 1.

Test each independent path at least once, including valid and invalid input routes.
For each loop, test zero iterations, one iteration, typical iterations, the maximum supported count, and one beyond the limit where meaningful.

Test nested-loop termination, early exits, break/continue logic, empty collections, and errors during iteration.
Record path identifiers and expected results; use branch/path coverage reports and regression tests.

This strategy is more informative than statement coverage because it checks alternate control-flow routes and loop termination. Full path coverage may be infeasible for unbounded loops, so bounded, independent, and safety-critical paths should be justified and prioritized.

QUESTION 5: Financial Application: Hybrid Black-Box and White-Box Testing

Production defects despite using both methods indicate that the techniques may have been applied independently, with weak requirements traceability, insufficient boundary tests, incomplete CFG/path coverage, or coverage metrics treated as proof of correctness. A hybrid strategy should connect external business rules to internal control-flow routes.

Black-box ECP: partition amount, account status, balance, currency, transaction type, and malformed input into valid and invalid classes.

Black-box BVA: test minimum/maximum transfer or payment values and adjacent values, including zero, negative amounts, rounding and balance limits.

Decision tables: combine account status, sufficient funds, daily limit, authentication status, and transaction type.

White-box CFG: map validation, authorization, debit, credit, commit, rollback, timeout, and exception paths.

Basis path testing: identify independent paths and test successful transactions, validation rejection, authorization failure, debit/credit failure, timeout, and rollback.
Integration and non-functional tests: concurrency, duplicate requests, idempotency, performance, access control, audit trails, and recovery.

Track statement and branch coverage, selected path coverage, requirements coverage, and defect leakage. A high percentage is not enough unless each test has meaningful assertions and the expected financial state is verified. Every production defect should become a regression test. This combined method detects invalid inputs, boundary errors, internal logic defects, and failures between services, reducing the chance of repeat incidents.

QUESTION 6: HR Payroll Bonus at 5, 10 and 15 Years

Incorrect payouts exactly at 5, 10, and 15 years strongly suggest boundary or comparison errors—for example, using greater-than instead of greater-than-or-equal, rounding service duration incorrectly, or applying the wrong bonus band at a threshold. Testing only typical service durations would miss these errors.

Assume the business specification defines thresholds at 5, 10 and 15 years. For each threshold T, test $T-\varepsilon$, T, and $T+\varepsilon$, where ε is the smallest meaningful service-time unit (for example, one day if service is calculated by date). Also test immediately before and after the annual anniversary and the minimum/maximum supported service period.

Threshold	Test points	Purpose
5 years	4y 364d; 5y; 5y + 1d	Check entry into the 5-year band
10 years	9y 364d; 10y; 10y + 1d	Check transition into the 10-year band
15 years	14y 364d; 15y; 15y + 1d	Check transition into the 15-year band

ECP should separate service bands and performance ratings (for example, below target, meets target, exceeds target—using the actual payroll policy). Test each meaningful combination, including invalid or missing rating, part-time/leave rules if relevant, and employees whose anniversary falls on a month boundary. Expected bonus amounts must be calculated independently from the approved policy.

Exploratory testing is useful for discovering unexpected issues but does not guarantee repeatable coverage of every threshold. BVA and ECP are superior for this specific deterministic financial logic because they systematically target transition points and classes. Retain exploratory testing as an additional check, and verify payroll totals, rounding, audit records, and regression results.

QUESTION 7: Laboratory Temperature Control: CFG and 100% MC/DC

The alarm failure may be caused by an untested combination of sensor status, temperature threshold, heater/cooler state, or alarm-enable condition. Ordinary branch coverage may execute both outcomes of a decision without proving that each Boolean condition independently affects the decision.

Conceptual CFG:
START → Read sensor → [Sensor valid?]
No → Disable unsafe control → Trigger alarm → END
Yes → [Temperature > upper limit?]
Yes → Cooler ON; heater OFF
No → [Temperature < lower limit?]
Yes → Heater ON; cooler OFF
No → Heater OFF; cooler OFF
→ [Alarm condition met?] → Yes: Alarm ON / No: Alarm OFF → END

For MC/DC, each atomic condition in a compound decision must be shown to independently change that decision's outcome while other conditions are held fixed. For example, if alarm = sensorFault OR (temperatureHigh AND alarmEnabled), test cases should demonstrate the independent effect of sensorFault, temperatureHigh, and alarmEnabled. The exact test vectors must follow the implemented Boolean expression and safety requirements.

Build a CFG including sensor read failures, stale readings, threshold branches, control actions, and alarm handling.
List every Boolean condition and decision; derive a minimal MC/DC test set and document which pair demonstrates each independent effect.
Test values just below, at, and above temperature thresholds; sensor disconnected, stuck, delayed, and out-of-range readings.

Verify fail-safe behaviour: a sensor fault must not silently suppress a required alarm or leave heater/cooler outputs unsafe.

Measure branch coverage and MC/DC coverage separately, then run hardware-in-the-loop tests and fault injection. 100% MC/DC for the relevant decisions gives stronger evidence than statement or branch coverage alone, but it does not by itself prove system safety; requirements-based tests, timing checks, and independent review are also required.

QUESTION 8: Travel Booking Promo Codes: Decision Table Testing

Testing promo codes one at a time misses interactions between promotions, user eligibility, and the booking price. A stacking or ordering defect may apply two discounts incorrectly and produce a negative total. The system should enforce explicit rules for eligibility, stacking, discount caps, and a minimum total of zero.

Illustrative decision table (the actual portal rules must define which promo types can stack):

Rule	Price valid?	Promo eligible?	User tier eligible?	Stacking allowed?	Expected result
R1	No	Any	Any	Any	Reject booking/input
R2	Yes	No	Any	Any	Ignore/reject promo with clear message
R3	Yes	Yes	No	Any	Reject tier-restricted promo
R4	Yes	Yes	Yes	No	Apply only the permitted single promo
R5	Yes	Yes	Yes	Yes	Apply permitted combined discount, then enforce floor at zero

Cause-effect mapping can express causes such as valid price, eligible promo, matching user tier, and stacking permission, with effects such as accept promo, reject promo, calculate total, and show validation error. Include tests for zero and small positive prices, maximum discount, duplicate codes, expired codes, incompatible codes, premium/basic/free tiers, and different promo orderings.

An optimized set contains at least one test for each decision-table rule plus boundary and interaction cases. Where combinations are numerous, pairwise selection can reduce redundancy, but all high-risk combinations and all rules that can cause a negative total must be tested explicitly. Assert that total = max(0, subtotal − valid discounts + applicable charges), according to the specification, and verify that rejected codes do not change the booking.

QUESTION 9: Database Migration Nested Loops and Cyclomatic Complexity

High statement coverage only shows that statements executed; it does not show that loop exit conditions, nested-loop interactions, or termination logic are correct. An infinite loop can occur when an index is not updated, a termination condition is wrong, or an error path returns to the loop without progress.

Construct a CFG for record iteration, field iteration, validation decisions, error handling, and database write/retry logic. Identify independent paths using $V(G) = E - N + 2P$ (for one connected CFG, $E - N + 2$). The result gives the number of independent basis paths to design for, not the total number of possible paths.

Test an empty database (zero loop iterations), one record, multiple records, and the maximum supported batch.

For nested loops, test zero, one, and multiple fields per record; malformed records; duplicate keys; and a failure in the inner loop.

Exercise each loop exit and continue/break path, including retry limits and exceptions.
Add a watchdog/timeout and assert progress counters or cursor movement to detect non-termination.

Use large datasets, mixed valid/invalid records, and deterministic fixtures; verify record counts and data integrity after migration.
Basis Path Testing provides a systematic set of linearly independent routes, while loop-boundary tests specifically check termination. Cyclomatic complexity helps estimate structural testing effort and identify overly complex routines. It must be combined with runtime timeouts, performance tests, and data-reconciliation checks to prevent infinite loops and silent migration corruption.

QUESTION 10: Food Delivery Real-Time Tracker: State Transition Testing

ECP and BVA focus on input values and boundaries; they may miss invalid sequences of events. The reported stuck state after driver cancellation indicates that the transition model does not handle cancellation from every permitted state or does not define a recovery state.

Expected state model:
Ordered → Preparing → Out for Delivery → Delivered
| | |
└──────────┴──────────────────────────┴──→ Cancelled (if policy permits)
Out for Delivery → Driver Cancelled → Reassign Driver → Out for Delivery
Any active state → System/Network Error → Retry or Support Escalation

Create a state transition table with current state, event, guard, next state, and action. Test every valid transition and important invalid transition. Scenarios include customer cancellation before preparation, restaurant rejection, driver cancellation while en route, reassignment success/failure, network loss and recovery, duplicate status events, delivery completion, and cancellation after delivery (which should be rejected or handled as a support/refund process).

Check that the UI, backend order record, notifications, payment/refund, and driver assignment remain consistent. Test transition pairs or sequences, not just each state individually. State Transition Testing directly addresses event ordering and missing transitions, making it essential for dynamic workflows.

QUESTION 11: Cybersecurity Authentication: Error Guessing and Path Analysis

Detected brute-force vulnerabilities suggest the tests may have verified successful login flows without adequately testing repeated failures, OTP expiry, lockout behaviour, rate limits, or interactions between password, OTP, and IP checks. Black-box testing checks observable security rules, while white-box path analysis checks whether the implementation can bypass or incorrectly order those rules.

Error-guessing cases include repeated wrong passwords, repeated OTP guesses, expired/reused OTPs, OTP replay, concurrent login attempts, account enumeration, password reset abuse, missing fields, malformed requests, and switching IP/device during authentication. Execute only in an authorized test environment.

Build a CFG for: receive credentials → validate format → check rate limit/account status → verify password → verify OTP and expiry → evaluate IP/risk policy → grant or deny access → log event/update counters. Test independent paths for valid login, bad password,

bad/expired OTP, blocked account, risky IP, rate-limit exceeded, service failure, and recovery. Verify that no error or fallback path grants access without all required factors.

Apply rate limiting per account and source, progressive delays or lockout with safe recovery, and monitoring/alerting.
Use generic error messages to reduce account enumeration and secure, tamper-resistant audit logs.
Check that rate-limit counters cannot be bypassed by concurrency, session changes, or alternate endpoints.
Use branch/path coverage and security tests against requirements; review cryptographic and session-handling practices.

The hybrid strategy finds both unexpected abuse cases and structural bypasses. Coverage metrics support the analysis but do not prove security; penetration testing, threat modelling, code review, and regression testing are also necessary.

QUESTION 12: E-learning Subscription Expiration and Upgrade Dates
An error exactly on the expiration date suggests ambiguity about whether the expiry date is inclusive, the timezone used, and whether upgrades take effect before or after entitlement evaluation. The product specification must define these rules before expected results can be finalized.

Use ECP for date categories: before expiration, exactly at the expiration boundary, after expiration, before subscription start, and invalid/missing date. For subscription type, test Free, Basic, and Premium; for course type, test courses available to all tiers and courses restricted to higher tiers. Test the meaningful combinations of date class × current tier × requested tier × course entitlement.

Date class	Example	Expected behaviour to define/test
Before expiry	Expiry date/time − 1 unit	Current entitlement remains active
At expiry	Exact expiry instant / local date boundary	Apply documented inclusive/exclusive rule consistently
After expiry	Expiry + 1 unit	Expired benefits removed or downgraded as specified
Upgrade at expiry	Upgrade request at exact boundary	No gap or unauthorized access; deterministic tier result
Invalid date	Missing, malformed, inconsistent dates	Reject safely and log issue

Include timezone and daylight-saving handling if applicable, simultaneous upgrade/expiry requests, payment success/failure, and repeated requests. Verify access permissions, billing, course availability, and user-visible status. ECP reduces redundant testing by selecting representative cases from equivalent date and tier classes, while boundary-focused cases target the known failure. A decision table can cover interactions and confirm seamless transitions.

QUESTION 13: Smart-Home Lighting: Branch and Condition Coverage

Statement coverage can execute every line while leaving a switch case or a Boolean outcome untested. This is risky when sensor readings control physical hardware, because an omitted branch may cause excessive brightness, darkness, flicker, or unsafe device behaviour.

For each switch statement, test every case and the default case, including invalid sensor values. For each Boolean expression, test conditions both True and False and verify the decision outcome. Example: if (sensorValid AND ambientLightLow) then increase brightness; otherwise use a safe fallback. Test sensorValid=True/False and ambientLightLow=True/False so that each condition and each decision outcome is exercised.

Create a CFG covering sensor acquisition, calibration, switch cases, Boolean decisions, dimming calculations, actuator commands, and error handling.
Apply branch coverage to every switch/if outcome and condition coverage to each atomic Boolean condition.

Test thresholds just below, at, and above dimming levels; missing, noisy, saturated, and disconnected sensors.
Use hardware-in-the-loop tests to compare requested brightness with actual light output; test startup, shutdown, communication delay, and actuator failure.

Branch and condition coverage reveal missing logical outcomes that statement coverage can hide. Combine them with boundary, integration, and regression tests. Coverage should be reviewed alongside requirements and physical output measurements to reduce hardware-software integration failures.

QUESTION 14: Logistics Shipping Weight: Robust Boundary Value Analysis

The missed zero and negative weights show that input validation did not include invalid boundary values. If the minimum valid weight is Wmin and maximum is Wmax, tests should include values below, at, and above both limits. Zero and negative weights must not be silently accepted or used to calculate shipping charges.

Input	Expected result
Negative weight (−ε)	Reject; no shipping quote/order creation
Zero weight	Reject unless the specification explicitly supports a zero-weight shipment
Wmin − ε	Reject
Wmin	Accept and calculate according to rate rules
Typical valid weight	Correct rate
Wmax	Accept if within service limit
Wmax + ε	Reject or route to special handling per specification
Blank, text, overflow, excessive decimals	Validation error without crash

This is robust BVA because it includes values outside the valid range as well as boundary values. Also test valid weight with different destination zones and delivery speeds, since shipping price is based on all three fields. Confirm that invalid inputs cannot bypass validation through API calls, edited requests, or alternate screens.

Negative testing is necessary because real users, integrations, and malicious requests can supply invalid data. It prevents incorrect charges, invalid labels, downstream carrier failures, and inconsistent records. Validate server-side, return clear messages, log abnormal attempts, and regression-test each discovered defect.

QUESTION 15: Medical Imaging Parallel Processing: Basis Path Testing

Inconsistent results for prime-sized datasets suggest that partitioning, chunk allocation, remainder handling, or thread synchronization may fail when data cannot be divided evenly among workers. Random testing is unlikely to systematically exercise all relevant control-flow and size boundaries.

Construct a CFG for input validation, worker-count selection, partition calculation, per-thread processing, remainder processing, result aggregation, and error handling. Use basis path testing to identify independent paths, with V(G) = E − N + 2P. Select tests for each independent route and separately design data-size tests around the parallel partition logic. Test dataset sizes 0, 1, small primes, larger primes, even numbers, powers of two, and values just below/at/above multiples of the worker count.

Vary thread counts, including one thread, fewer items than threads, and more items than threads.

Check remainder handling, empty partitions, race conditions, synchronization, deterministic repeatability, and thread failure.

Compare parallel results with a trusted serial reference implementation and verify numerical tolerances, output dimensions, and checksums where appropriate.

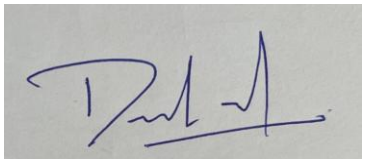
Repeat tests under different schedules and loads; use sanitizers/race detection tools when available.

Basis Path Testing provides structural coverage of independent control-flow routes, while targeted prime/composite sizes test the mathematical partitioning risk. Randomized tests remain useful as a supplement, especially with reproducible seeds, but must not replace structured cases. Correctness comparison, stress testing, and concurrency testing together improve mathematical accuracy and system stability.

Code of Conduct:

*I **Deepa preya H / 192421047** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification